Simulated Bitcoin Exchange

Final Project · Phase 3

Andrés Proaño · Pablo Alvarado

CMP-5001-202520 · Aplicaciones Distribuidas
Universidad San Francisco de Quito
Semestre 202520

> **Objetivo:** simular un exchange de Bitcoin con minería on-chain, trading interno entre clientes y un historial auditable por cuenta.

1. Conceptos base

Blockchain
Cadena distribuida de bloques. Estado compartido entre nodos.

Node
Bitcoin Core ejecutando regtest, valida y propaga bloques/tx.

Wallet
Conjunto de claves y direcciones controladas por un actor.

Miner
Resuelve PoW para anexar bloques y reclama el coinbase.

Proof of Work
Reto criptográfico que asegura el orden de la cadena.

Coinbase maturity
El BTC del coinbase queda bloqueado **100 bloques** antes de poder gastarse.

> El BTC **nace en blockchain mediante minería** y debe madurar antes de poder gastarse o transferirse.

2. Dos mundos del sistema

Blockchain system

- 2 nodos Bitcoin Core (regtest)
- 10 mineros Python competitivos
- Wallets controladas por mineros
- Bloques, txs y maduración reales

⇄

Exchange system

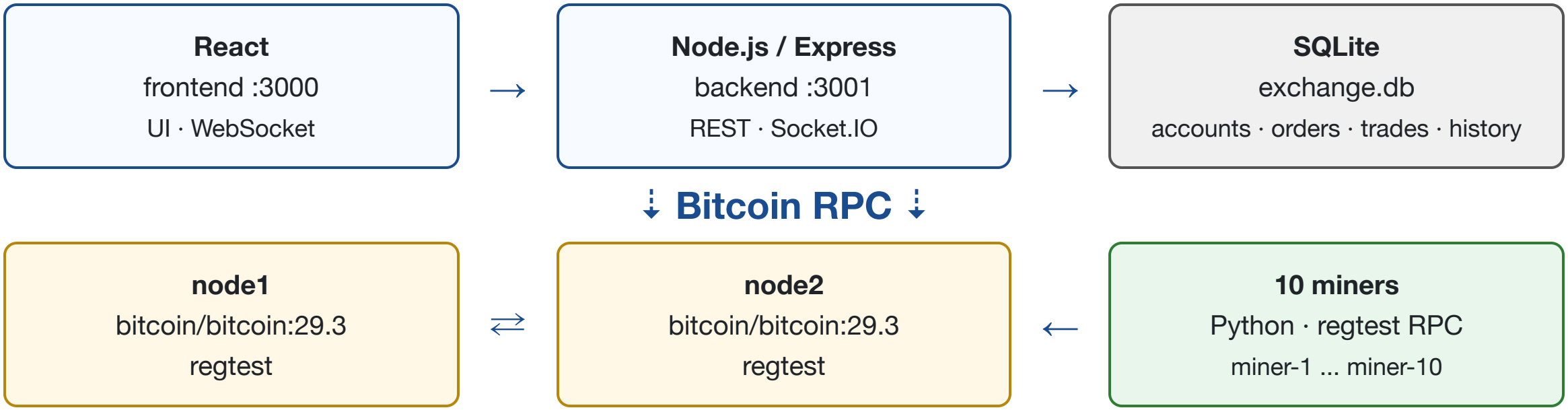
- SQLite: cuentas internas
- 30 clientes con $30,000 USD
- Owner account para fees
- Trading interno (BTC ↔ USD)

⬆ **frontera** ⬆

BTC entra al exchange únicamente cuando un miner emite una transacción on-chain hacia el *exchange deposit address* y el backend verifica la recepción.

> BTC en una wallet minera **no** es BTC dentro del exchange.

3. Arquitectura



> Cada servicio es un contenedor Docker. **Docker Compose** orquesta los 14 servicios. Imágenes publicadas multi-arch: `byandyx/btc-{backend,frontend,miner}:proyecto-final`

4. Fase 1 · Mining & exchange entry

1. Mine miner-X corre PoW en regtest y anexa un bloque

→

2. Mature Coinbase espera 100 bloques de confirmación

→

3. Threshold Si `btc_mature ≥ threshold`, dispara depósito

↓

4. On-chain tx miner llama `sendtoaddress` al deposit address del exchange

→

5. Backend verifica `POST /api/exchange/deposit` + `gettransaction` en node1

→

6. Cuenta creada Aparece `miner-X` en exchange con BTC available

> Fase 1 explica **cómo el BTC entra al sistema del exchange**: minería real, maduración respetada y verificación on-chain antes de acreditar saldo interno.

5. Fase 2 · Trading interno

client_1 Place BUY 0.5 BTC @ $50k
Reserva $25,000 USD

Matching engine price-time priority
full match / no match

miner-5 Place
SELL 0.5 BTC @ $50k Reserva 0.5 BTC

↓ trade ejecutado ↓

Buyer recibe 0.49 BTC (neto)
$25,000 debitado

Seller recibe $25,000 USD 0.5 BTC debitado

Owner fee 2% 0.01 BTC acreditado sobre el bruto del trade

Fase 2 convierte el sistema en un **mercado interno BTC ↔ USD** con reservas, prioridad price-time, matching atómico y fee del owner del 2 %.

6. Fase 3 · Historial por cliente

Cada cuenta acumula sus eventos BTC en la tabla `client_btc_history`:

Account ID	Client Name	Timestamp	Event Type	Event Details
client_1	Client 1	2026-05-04 16:56	**Buy BTC**	Received 0.49 BTC at $50,000 per BTC
miner-5	Miner 5	2026-05-04 16:56	**Sell BTC**	Sold 0.50 BTC at $50,000 per BTC
client_1	Client 1	2026-05-04 16:58	**Transfer BTC**	Transferred 0.10 BTC to bcrt1q4d… (txid f545…dcb9)

Buy BTC Cantidad + precio por BTC. Insertado al matchear orden compradora.

Sell BTC Cantidad + precio por BTC. Insertado al matchear orden vendedora.

Transfer BTC Cantidad retirada del exchange + dirección destino + txid on-chain.

> Fase 3 agrega **trazabilidad completa**: el orden cronológico permite explicar y auditar cada movimiento BTC de cualquier cuenta.

7. BTC Out · Retiro on-chain

UI Selecciona cuenta + address + monto

→

Backend `POST /api/clients/:id/transfer-btc` Valida fondos + address regtest

→

Bitcoin RPC `sendtoaddress` desde wallet exchange

↓

Blockchain Tx confirmable con `gettransaction <txid>`

→

UI Refresh vía WebSocket `history-updated`

→

Exchange DB Debita `btc_available` Inserta evento `TRANSFER_BTC` con txid + address

> BTC Out **saca BTC del exchange** mediante una transacción real en Bitcoin Core y deja un evento `Transfer BTC` auditable con su txid.

8. Demo · ciclo completo

1. **Servicios up** — `docker compose pull && docker compose up -d` (14 contenedores)
2. **Threshold = 1** y **Start Mining** desde el frontend
3. **Esperar deposit** — primer miner hace tx on-chain → cuenta `miner-X` con BTC
4. **SELL** desde miner-5 (0.5 BTC @ $50k) y **BUY** desde client_1 (0.5 BTC @ $50k)
5. **Match** instantáneo — verificar en *Completed Trades*
6. **Owner fee** sube a 0.01 BTC (= 2 % del bruto)
7. **History** del buyer muestra `Buy BTC 0.49`, del seller `Sell BTC 0.50`
8. **Transfer BTC Out** del buyer hacia un address regtest nuevo
9. **History** del buyer muestra `Transfer BTC` con txid real
10. **Verificación on-chain** — `bitcoin-cli gettransaction <txid>` lo confirma

> **El BTC nace, entra al exchange, se negocia, queda registrado y sale.** Todo el ciclo es observable en tiempo real desde la UI y verificable en la blockchain.

9. Cierre · valor del sistema

Phase 1 Blockchain real, mining competitivo, maduración y entrada al exchange verificada on-chain.

Phase 2 Mercado interno con balances reservados, matching price-time y fee del owner.

Phase 3 Trazabilidad por cuenta + retiro on-chain. Historial completo y auditable.

Entregables

- Código fuente en D2L + `README.txt` con miembros, contribuciones y comandos
- Imágenes Docker Hub multi-arch (`linux/amd64`, `linux/arm64`):
  - `byandyx/btc-backend:proyecto-final` · `:latest`
  - `byandyx/btc-frontend:proyecto-final` · `:latest`
  - `byandyx/btc-miner:proyecto-final` · `:latest`
- Ejecución de grading: `docker compose pull && docker compose up -d` (sin build local)

> El proyecto integra **blockchain**, **exchange interno** y **auditoría por historial** en un sistema coherente, contenedorizado y reproducible.

Gracias

Preguntas

Andrés Proaño · Pablo Alvarado

Repositorio · `byandyx/btc-{backend,frontend,miner}:proyecto-final`